

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



COMPUTER GRAPHICS

Assignment 1

Final Report

Instructor(s): Trần Thị Ngọc Trâm

Group: 5

Students: Lâm Mỹ Phương - 2352950
Lý Trần Tân -

HO CHI MINH CITY, OCTOBER 2026



Contents

1	Introduction	2
2	Overall Architecture and Shared Components	3
2.1	Technology and repository layout	3
2.2	Setup, build and run	4
2.3	Shared components	5
3	Part 1: Drawing Basic Shapes	5
3.1	Architecture and Components	6
3.2	Geometry: Vertices and Indices of Every Shape	7
3.3	Surface $z = f(x, y)$ and Imported Models	14
3.4	Rendering Modes and Shaders	15
3.5	Object Transformations	18
3.6	User Interface and User Instructions	19
3.7	Results	20
3.8	Testing	22
3.9	Limitations	22
4	Part 2: Visualization of Atoms and Molecules	23
4.1	Architecture and Components	23
4.2	Visualizing Atoms: Bohr's Model	23
4.3	Visualizing Molecules: Ball-and-Stick Model	23
4.4	Animation (Optional)	23
4.5	Rendering and Lighting	23
4.6	User Interface and User Instructions	23
4.7	Results	23
4.8	Limitations	23
5	Conclusion	23
5.1	Part 1	23
5.2	Part 2	24
6	Source Code	24
	References	25



1 Introduction

This report describes Assignment 1 of the Computer Graphics course (CO3059): *2D and 3D object rendering*. Students implement two interactive graphical applications with modern, shader-based OpenGL:

- **Part 1 – Drawing Basic Shapes:** render a variety of 2D and 3D shapes (including a user-defined surface and imported models) with a GUI, camera control, geometric transformations and several rendering modes.
- **Part 2 – Visualization of Atoms and Molecules:** visualize atoms with Bohr's model and simple molecules with the ball-and-stick model.

Both applications are written in C++17 on top of OpenGL 3.3 (core profile), GLFW, GLEW, GLM and Dear ImGui [1, 4, 5, 6, 7], run on Linux, and share a small common code base.

Organisation of this report. Section 2 describes what is common to both parts (technology, repository, build instructions, shared classes, lighting model). Sections 3 and 4 describe Part 1 and Part 2. Section 5 concludes and gives the link to the source code.



Requirements and where they are met.

Requirement (assignment text)	Implementation
<i>Part 1</i>	
2D shapes: triangle, rectangle, pentagon, regular hexagon, circle, ellipse, trapezoid, star, arrow	Section 3.2: generators in <code>Geometry.cpp</code>
3D solids: cube, sphere, cylinder, cone, truncated cone, tetrahedron, torus, prism	Section 3.2
Surface $z = f(x, y)$ from a user-provided function	Section 3.3.1: expression parser + grid mesh
Imported model from <code>.obj</code> / <code>.ply</code>	Section 3.3.2: hand-written loaders
GUI with menus / toolbars to select the shape to add	Section 3.6: menu bar and inspector (Dear ImGui)
Mouse and/or keyboard zoom, pan, rotate	Section 2.3.2 (camera) and Section 3.6 (controls)
Flat color, Gouraud, Phong, texture, wireframe	Section 3.4: one shader program, five modes
Translation, rotation, scaling	Section 3.5
Clean, modular code	Section 3.1
<i>Part 2</i>	
GUI with menus / toolbars to toggle the features	
Visualize atoms using Bohr's model: electrons as small spheres orbiting the nucleus in distinct shells	
Visualize simple molecules (e.g. H_2O , CO_2): atoms as colored spheres, bonds as cylinders	
Optional: animate electron motion along orbitals; molecular vibration or rotation	
Camera control and screenshots of the molecule structures	

2 Overall Architecture and Shared Components

2.1 Technology and repository layout

- **Language / API:** C++17, OpenGL 3.3 core profile, GLSL 330, 4× MSAA.
- **Libraries:** GLFW (window, input), GLEW (extension loading), GLM (math), Dear ImGui (GUI). Part 1 additionally uses the single-header libraries `stb_image`



/ stb_image_write (image loading and screenshots) [8].

- **Shared code:** the Shader and Camera classes and the ImGui / GLM libraries live in common/ and are compiled by both parts.

```
cg/
|-- Makefile, README.md      build both parts; overview
|-- common/                  shared by both parts
|   |-- src/                  Shader.h/.cpp, Camera.h/.cpp
|   `-- vendor/              imgui, glm
|-- part1_shapes/            Part 1
|   |-- src/ shaders/        application source, GLSL
|   |-- assets/              sample textures (png) and models (obj, ply)
|   |-- vendor/stb/          stb_image.h, stb_image_write.h
|   `-- tests/               unit tests (no GPU needed)
|-- part2_molecules/         Part 2
|   `-- src/ shaders/        application source, GLSL
|-- docs/                    assignment statement
`-- report/                  this report (.tex, .pdf) and its figures
```

Each part has its own Makefile and builds into its own build/ folder; the top-level Makefile calls both.

2.2 Setup, build and run

```
sudo apt install build-essential libglfw3-dev libglew-
dev      # Ubuntu / Debian
```

```
cd cg
make          # build both parts (or: make part1 / make part2)
make run1     # start Part 1 (make run2: Part 2)
make test     # Part 1 unit tests (no GPU needed)
make clean    # remove the build output of both parts
```

Dear ImGui, GLM and the stb headers are bundled in the repository, so no other dependency has to be installed. The programs must be started from their own folder (part1_shapes or part2_molecules) so that shaders/ (and, for Part 1, assets/) are found; make run1 / make run2 do this. All data files Part 1 needs to run (sample textures and models) are delivered in part1_shapes/assets/.

2.3 Shared components

2.3.1 Shader class

Shader (`common/src/Shader.cpp`) loads a vertex and a fragment shader from text files, compiles each stage and links the program; compile or link errors are turned into an exception carrying the GLSL info log. `use()` activates the program and the typed setters `setMat4`, `setMat3`, `setVec3`, `setFloat`, `setInt`, `setBool` upload uniforms by name. Both parts keep their GLSL files in their own `shaders/` folder.

2.3.2 Camera

The Camera class (`common/src/Camera.cpp`) is an orbit camera defined by `yaw`, `pitch`, `distance` and a `target`:

$$\text{eye} = \text{target} + d (\cos \varphi \cos \psi, \sin \varphi, \cos \varphi \sin \psi),$$

with $\psi = \text{yaw}$ and $\varphi = \text{pitch}$ (clamped to $\pm 89^\circ$). The view matrix is `lookAt(eye, target, up)` and the projection is a perspective matrix (default field of view 45° , near 0.1, far 100, adjustable). The class also implements the mouse interaction used by both applications: dragging the left button changes yaw / pitch (orbit), dragging the right button moves the target in the camera's right / up plane (pan, scaled by the distance), and the wheel changes the distance (zoom, clamped to $[1, 50]$). The applications ignore the mouse while it is over the ImGui panels.

2.3.3 Phong lighting model

Both applications light their objects with the Phong reflection model [2] and a single point light whose position and ambient / diffuse / specular colors are editable in the GUI. For a surface point P with unit normal N , light direction $L = \text{normalize}(\text{lightPos} - P)$, view direction $V = \text{normalize}(\text{viewPos} - P)$ and reflection $R = \text{reflect}(-L, N)$, the color is

$$C = L_a \odot m_a + L_d \odot m_d \max(N \cdot L, 0) + L_s \odot m_s \max(R \cdot V, 0)^n,$$

where L_a, L_d, L_s are the light's ambient, diffuse and specular colors, m_a, m_d, m_s the material's, $\odot$ is the component-wise product and n is the shininess. The ambient term keeps the unlit side visible, the diffuse term gives the object its volume, and the specular term adds the bright highlight whose size is controlled by n . The way each part evaluates this formula (per vertex, per fragment) is described in its own section.

3 Part 1: Drawing Basic Shapes

Part 1 asks for an interactive application that renders basic 2D shapes (triangle, rectangle, pentagon, regular hexagon, circle, ellipse, trapezoid, star, arrow) and 3D shapes



(cube, sphere, cylinder, cone, truncated cone, tetrahedron, torus, prism, a surface $z = f(x, y)$ and an imported `.obj` / `.ply` model), with a GUI to choose the shape, mouse / keyboard zoom, pan and rotate, and five rendering modes (flat color, Gouraud, Phong, texture, wireframe).

3.1 Architecture and Components

3.1.1 Modules

The code is split so that everything that does not need OpenGL (geometry, parsing, file loading) is independent of the rendering code and can be unit-tested without a window.

File(s)	Responsibility
<code>main.cpp</code>	Window, GLFW callbacks (mouse / keyboard), render loop, command-line options.
<code>App.h</code>	State shared by <code>main</code> and the GUI: <code>Scene</code> , <code>Camera</code> , light, camera presets.
<code>Gui.cpp</code>	Menu bar, object list, parameter / transform / appearance editors, import dialog.
<code>Scene.cpp</code>	Object list, selection, duplicate / delete, ground grid, axes, selection box, demo scene.
<code>SceneObject.cpp</code>	One shape: kind + parameters, transform, appearance; builds its mesh; issues the draw call.
<code>Geometry.cpp</code>	CPU mesh generators for all 2D / 3D shapes and mesh utilities (normals, colors, edge list). No OpenGL.
<code>ShapeMesh.cpp</code>	GPU buffers (VAO / VBO / two EBOs); <code>draw()</code> and <code>drawWire()</code> .
<code>Surface.cpp</code> , <code>ExprParser.cpp</code>	Mesh of $z = f(x, y)$ and the formula compiler.
<code>ModelLoader.cpp</code>	<code>.obj</code> and <code>.ply</code> importers.
<code>Texture.cpp</code>	Image loading (stb_image), mipmaps, per-file cache, built-in checkerboard.
<code>Screenshot.cpp</code> , <code>Paths.cpp</code>	PNG screenshot; locating shader / asset files.
<code>common/Shader.cpp</code>	Compiles and links GLSL, uniform setters.
<code>common/Camera.cpp</code>	Orbit camera: view / projection matrices, mouse rotate / pan / zoom.

3.1.2 Data flow

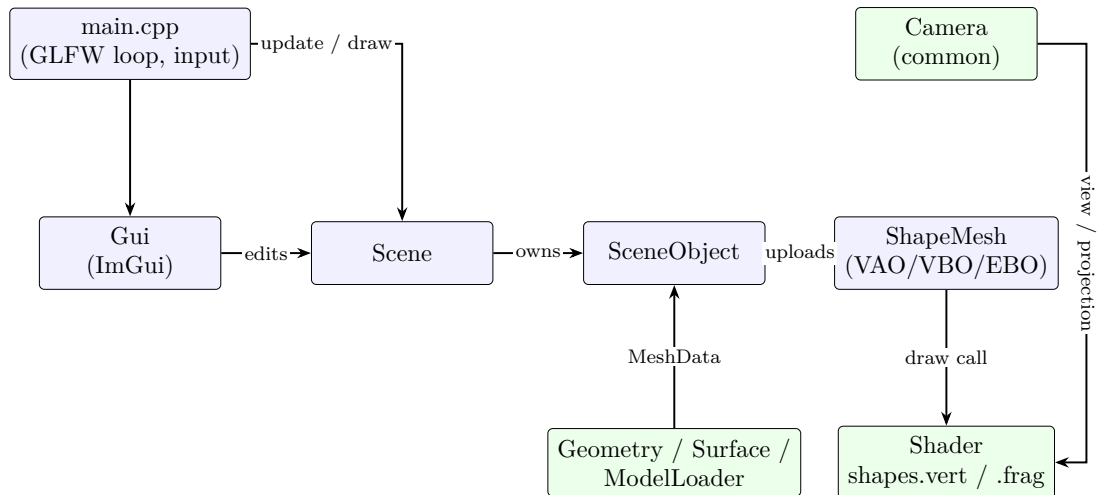


Figure 1: Main components (*blue*: application classes, *green*: data sources / GPU side). A **SceneObject** asks **Geometry** (or **Surface** / **ModelLoader**) for a **MeshData**, uploads it as a **ShapeMesh**, and every frame sets its uniforms and draws it with the shared shader.

Each frame, **main** (i) processes input, (ii) uploads the camera and light uniforms, (iii) lets the **Scene** draw the grid, every visible object and the selection box, (iv) draws the **ImGui** interface on top.

3.2 Geometry: Vertices and Indices of Every Shape

3.2.1 Conventions

Vertex format. Every mesh uses the same interleaved vertex (11 floats = 44 bytes):

Attribute	Type	Used for
position (location 0)	vec3	object-space position
normal (location 1)	vec3	unit normal, lighting
color (location 2)	vec3	per-vertex color (Gouraud mode, grid, axes)
uv (location 3)	vec2	texture coordinates

Index buffers. Each mesh owns two index lists (both `unsigned int`):

- **indices:** a *triangle list*, 3 indices per triangle, drawn with `GL_TRIANGLES`. Triangles are wound counter-clockwise when seen from outside (the direction of the vertex normal).



- **wireIndices:** a *line list*, 2 indices per edge, drawn with `GL_LINES` in wireframe mode (Section 3.2.10).

Notation. V = number of vertices, I = number of triangle indices, $T = I/3$ = number of triangles, E = number of wireframe edges. All shapes are centred at the origin and are about one unit large; 2D shapes lie in the XY plane with normal $+Z$, 3D solids use Y as the up axis.

Why vertices are duplicated. A vertex is the unit stored in the vertex buffer: one position *together with* one normal, one color and one UV. Whenever two faces meet along a sharp edge (cube, tetrahedron, prism sides, cylinder caps) each face needs its own normal, so the corner is stored once per face — e.g. a cube has $6 \times 4 = 24$ vertices, not 8. Likewise the first and last column of a sphere or torus have the same position but different u texture coordinates (0 and 1), so the seam is stored twice. Using indices lets the triangles of a face share the vertices of that face.

Parameters. The “default” columns below use the defaults of the GUI: 48 segments for round shapes, 24 stacks (sphere) / tube segments (torus), 6 prism sides, 5 star points, surface resolution 64.



3.2.2 Summary

Shape	Parameters	V	I	T	E	Defaults ($V/I/E$)
Triangle	$n = 3$	$n + 1$	$3n$	n	n	4 / 9 / 3
Rectangle	—	5	12	4	4	5 / 12 / 4
Pentagon	$n = 5$	$n + 1$	$3n$	n	n	6 / 15 / 5
Hexagon	$n = 6$	$n + 1$	$3n$	n	n	7 / 18 / 6
Circle	N segments	$N + 1$	$3N$	N	N	49 / 144 / 48
Ellipse	N segments	$N + 1$	$3N$	N	N	49 / 144 / 48
Trapezoid	—	5	12	4	4	5 / 12 / 4
Star	p points	$2p + 1$	$6p$	$2p$	$2p$	11 / 30 / 10
Arrow	—	7	15	5	7	7 / 15 / 7
Cube	—	24	36	12	12	24 / 36 / 12
Sphere	S sectors, K stacks	$(S+1)(K+1)$	$6S(K-1)$	$2S(K-1)$	$S(2K-1)$	1225 / 6624 / 2256
Cylinder	N segments	$4N + 6$	$12N$	$4N$	$3N$	198 / 576 / 144
Cone	N segments	$3N + 4$	$9N$	$3N$	$2N$	148 / 432 / 96
Truncated cone	N segments	$4N + 6$	$12N$	$4N$	$3N$	198 / 576 / 144
Tetrahedron	—	12	12	4	6	12 / 12 / 6
Torus	M ring, m tube	$(M+1)(m+1)$	$6Mm$	$2Mm$	$2Mm$	1225 / 6912 / 2304
Prism	n sides	$6n + 4$	$12n$	$4n$	$3n$	40 / 72 / 18
Surface	$N \times N$ cells	$(N+1)^2$	$\leq 6N^2$	$\leq 2N^2$	depends	4225 / 24576 / 11704 [†]
Model	from file	file	$3 \sum (k_f - 2)$	—	—	see Section 3.3.2

Table 1: Vertex, index, triangle and edge counts. These closed forms are derived in the following subsections and were checked against the real generators with a program that builds each shape for many parameter values. [†] for $f = \sin x \cos y$ on $[-3, 3]^2$.

3.2.3 2D shapes: triangle fans

All 2D shapes except the arrow are *convex or star-shaped around the origin*, so they are triangulated as a **triangle fan** around a centre vertex $c = (0, 0, 0)$: given a counter-clockwise ring of n boundary points $p_0, \dots, p_{n-1}$,

$$\text{vertices: } v_0 = c, \quad v_{1+i} = p_i \quad (i = 0, \dots, n-1) \quad \Rightarrow \quad V = n + 1,$$

$$\text{triangles: } (0, 1+i, 1+((i+1) \bmod n)), \quad i = 0, \dots, n-1 \quad \Rightarrow \quad I = 3n.$$

Every vertex has normal $(0, 0, 1)$. Texture coordinates use planar mapping of the bounding box: $u = \frac{x-x_{\min}}{x_{\max}-x_{\min}}$, $v = \frac{y-y_{\min}}{y_{\max}-y_{\min}}$. Only the ring changes between shapes:



Shape	Ring points p_i	n
Triangle, pentagon, hexagon, circle	$R(\cos \alpha_i, \sin \alpha_i)$ with $\alpha_i = \frac{\pi}{2} + \frac{2\pi i}{n}$, $R = 0.5$ (first vertex points up)	$3, 5, 6, N$
Ellipse	$(r_x \cos \alpha_i, r_y \sin \alpha_i)$, $\alpha_i = \frac{2\pi i}{N}$ (default $r_x = 0.5$, $r_y = 0.3$)	N
Rectangle $w \times h$	$(-\frac{w}{2}, -\frac{h}{2}), (\frac{w}{2}, -\frac{h}{2}), (\frac{w}{2}, \frac{h}{2}), (-\frac{w}{2}, \frac{h}{2})$	4
Trapezoid (b, t, h)	$(-\frac{b}{2}, -\frac{h}{2}), (\frac{b}{2}, -\frac{h}{2}), (\frac{t}{2}, \frac{h}{2}), (-\frac{t}{2}, \frac{h}{2})$	4
Star with p points	$r_i(\cos \alpha_i, \sin \alpha_i)$, $\alpha_i = \frac{\pi}{2} + \frac{\pi i}{p}$, $r_i = R$ for even i and ρR for odd i ($R = 0.5$, inner ratio $\rho = 0.4$)	$2p$

Example: triangle ($n = 3$, $V = 4$, $I = 9$).

index	x	y	role	
0	0	0	centre	
1	0	0.5	$\alpha = 90^\circ$	indices: (0, 1, 2), (0, 2, 3), (0, 3, 1)
2	-0.433	-0.25	$\alpha = 210^\circ$	
3	0.433	-0.25	$\alpha = 330^\circ$	

Example: rectangle 1×0.7 ($V = 5$, $I = 12$). Vertices $0 = (0, 0)$, $1 = (-0.5, -0.35)$, $2 = (0.5, -0.35)$, $3 = (0.5, 0.35)$, $4 = (-0.5, 0.35)$; indices (0, 1, 2), (0, 2, 3), (0, 3, 4), (0, 4, 1).

Arrow (concave, $V = 7$, $I = 15$). A fan from the centre would cover points outside the arrow, so the arrow is triangulated by hand. Seven shared vertices are used — sharing them is what makes the shaft/head seam an *interior* edge in wireframe mode (Section 3.2.10):

index	0	1	2	3	4	5	6
(x, y)	$(-0.5, -0.15)$	$(0.1, -0.15)$	$(0.1, 0.15)$	$(-0.5, 0.15)$	$(0.1, -0.4)$	$(0.5, 0)$	$(0.1, 0.4)$
part		shaft			head		

Indices: shaft (0, 1, 2), (0, 2, 3); head (4, 5, 1), (1, 5, 2), (2, 5, 6) — five triangles, 15 indices.

3.2.4 Cube

The cube of side $s = 1$ ($h = s/2$) is built face by face. Each face has an outward normal n and two tangent axes u, v with $u \times v = n$ (so the winding is counter-clockwise from outside). Its four vertices are

$$h(n-u-v), \quad h(n+u-v), \quad h(n+u+v), \quad h(n-u+v), \quad \text{UV } (0, 0), (1, 0), (1, 1), (0, 1),$$

all with normal n , and the two triangles are $(b+0, b+1, b+2)$ and $(b+0, b+2, b+3)$ where $b = 4f$ is the index of the face's first vertex. The six faces:

face	n	u	v
$+X$	$(1, 0, 0)$	$(0, 0, -1)$	$(0, 1, 0)$
$-X$	$(-1, 0, 0)$	$(0, 0, 1)$	$(0, 1, 0)$
$+Y$	$(0, 1, 0)$	$(1, 0, 0)$	$(0, 0, -1)$
$-Y$	$(0, -1, 0)$	$(1, 0, 0)$	$(0, 0, 1)$
$+Z$	$(0, 0, 1)$	$(1, 0, 0)$	$(0, 1, 0)$
$-Z$	$(0, 0, -1)$	$(-1, 0, 0)$	$(0, 1, 0)$

$V = 6 \cdot 4 = 24$, $I = 6 \cdot 6 = 36$. For example the $+Z$ face (indices 16–19) has positions $(-.5, -.5, .5)$, $(.5, -.5, .5)$, $(.5, .5, .5)$, $(-.5, .5, .5)$ and triangles $(16, 17, 18)$, $(16, 18, 19)$.

3.2.5 Sphere

A UV sphere of radius $r = 0.5$ with S sectors (longitude) and K stacks (latitude). Vertex (i, j) , $i = 0..K$, $j = 0..S$, is stored at index $i(S + 1) + j$:

$$\begin{aligned}\varphi_i &= \frac{\pi}{2} - \frac{\pi i}{K}, & \theta_j &= \frac{2\pi j}{S}, \\ n_{ij} &= (\cos \varphi_i \cos \theta_j, \sin \varphi_i, \cos \varphi_i \sin \theta_j), & p_{ij} &= r n_{ij}, \\ (u, v)_{ij} &= \left(\frac{j}{S}, 1 - \frac{i}{K}\right).\end{aligned}$$

For the normals of a sphere centred at the origin the normal equals the normalised position, so no extra computation is needed. There are $K + 1$ rows and $S + 1$ columns ($j = S$ duplicates $j = 0$ for the texture seam), hence $V = (S + 1)(K + 1)$.

Each quad between rows $i, i+1$ and columns $j, j+1$ is split in two triangles. With $k_1 = i(S + 1) + j$ and $k_2 = k_1 + S + 1$:

```
1 if (i != 0) { push(k1, k1 + 1, k2); } // skipped at the north pole
   (degenerate)
2 if (i != stacks - 1) { push(k1 + 1, k2 + 1, k2); } // skipped at the south pole
```

Counting: K rows of S quads give $2SK$ triangles, minus the S degenerate triangles at each pole: $T = 2SK - 2S$, so $I = 3T = 6S(K - 1)$. (Default $S = 48$, $K = 24$: $V = 1225$, $I = 6624$.)

3.2.6 Cylinder, cone and truncated cone

All three are the same generator `frustum( $r_b$ ,  $r_t$ ,  $h$ ,  $N$ )` with bottom radius r_b , top radius r_t , height h ($= 1$) and N segments (cylinder $r_t = r_b = 0.5$, cone $r_t = 0$, truncated cone $r_t = 0.25$ by default).

Side. For $i = 0..N$ with $a_i = 2\pi i/N$ two vertices are created (bottom at index $2i$, top at $2i+1$):

$$\text{bottom} = (r_b \cos a_i, -\frac{h}{2}, r_b \sin a_i), \quad \text{top} = (r_t \cos a_i, \frac{h}{2}, r_t \sin a_i),$$

$$n_i = \text{normalize}(h \cos a_i, r_b - r_t, h \sin a_i), \quad (u, v) = (i/N, 0) \text{ and } (i/N, 1).$$

The slanted normal (the y -component $r_b - r_t$) is what makes the cone and truncated cone shade correctly; for the cylinder it reduces to $(\cos a_i, 0, \sin a_i)$. For segment $i < N$ with $b_0 = 2i$, $t_0 = 2i + 1$, $b_1 = 2i + 2$, $t_1 = 2i + 3$: triangles (b_0, t_0, b_1) and (b_1, t_0, t_1) . So the side has $2(N + 1)$ vertices and $2N$ triangles (for the cone, N of them are degenerate because $t_0 = t_1$ is the apex; they are harmless and are ignored in wireframe).

Caps. A cap is added only if its radius is positive. A cap has its own vertices (its normal is $\pm Y$, not the side normal): one centre plus $N + 1$ ring vertices $(r \cos a_i, \pm \frac{h}{2}, r \sin a_i)$ with $uv = (\frac{1}{2} + \frac{1}{2} \cos a_i, \frac{1}{2} + \frac{1}{2} \sin a_i)$, and N triangles (c, p_i, p_{i+1}) for the bottom cap and (c, p_{i+1}, p_i) for the top cap.

Totals.

	V	T	I
cylinder / truncated cone (2 caps)	$2(N+1) + 2(N+2) = 4N + 6$	$2N + 2N = 4N$	$12N$
cone (bottom cap only)	$2(N+1) + (N+2) = 3N + 4$	$2N + N = 3N$	$9N$

3.2.7 Tetrahedron

A regular tetrahedron of edge $e = 1.2$ is inscribed in a cube: with $k = e/(2\sqrt{2}) \approx 0.4243$ its corners are

$$P_0 = k(1, 1, 1), \quad P_1 = k(1, -1, -1), \quad P_2 = k(-1, 1, -1), \quad P_3 = k(-1, -1, 1),$$

(every pair is $2\sqrt{2}k = e$ apart). Each of the 4 faces $(P_0P_1P_2)$, $(P_0P_3P_1)$, $(P_0P_2P_3)$, $(P_1P_3P_2)$ gets its own three vertices carrying the face normal $n = \text{normalize}((b - a) \times (c - a))$; if $n \cdot (a + b + c) < 0$ the normal points inward, so b and c are swapped and n is negated. UV are $(0, 0), (1, 0), (\frac{1}{2}, 1)$. Indices: face f uses $(3f, 3f+1, 3f+2)$. $V = 4 \cdot 3 = 12$, $I = 12$.

3.2.8 Torus

A torus with ring radius $R = 0.5$ and tube radius r (default 0.18), M segments around the ring and m around the tube. Vertex (i, j) , $i = 0..M$, $j = 0..m$ at index $i(m + 1) + j$, with $\theta_i = 2\pi i/M$, $\phi_j = 2\pi j/m$:

$$p_{ij} = ((R + r \cos \phi_j) \cos \theta_i, \quad r \sin \phi_j, \quad (R + r \cos \phi_j) \sin \theta_i),$$

$$n_{ij} = (\cos \phi_j \cos \theta_i, \quad \sin \phi_j, \quad \cos \phi_j \sin \theta_i), \quad (u, v) = (i/M, j/m).$$



The normal is the direction from the tube's centre circle to the surface point. With $a = i(m + 1) + j$ and $b = a + m + 1$ the quad is split into $(a, a+1, b)$ and $(b, a+1, b+1)$. $V = (M + 1)(m + 1)$, $T = 2Mm$, $I = 6Mm$.

3.2.9 Prism

A regular n -gon prism (radius 0.5, height 1, axis Y). Each side face i lies between angles $a_0 = 2\pi i/n$ and $a_1 = 2\pi(i + 1)/n$ and has its own four vertices (flat normal = $(\cos \frac{a_0+a_1}{2}, 0, \sin \frac{a_0+a_1}{2})$), in the order bottom- a_0 , top- a_0 , bottom- a_1 , top- a_1 with triangles $(0, 1, 2)$ and $(2, 1, 3)$ (offset by $4i$). The two caps are the same disc caps as for the cylinder, built with $N = n$. Therefore $V = 4n + 2(n + 2) = 6n + 4$ and $T = 2n + n + n = 4n$, $I = 12n$.

3.2.10 Wireframe edges

Drawing every triangle edge would show the diagonals of quads and the radial lines of the caps, while the assignment asks for “only the edges of the shape”. Therefore each mesh has a separate edge list `wireIndices`, built once by `buildWireIndices()`:

1. Vertices with the same position (quantised to 10^{-4}) get the same *canonical id*, so duplicated vertices of a cube corner or a texture seam are recognised as one point.
2. For every non-degenerate triangle, its face normal is computed and each of its 3 edges is recorded in a map keyed by the pair of canonical ids.
3. An edge is emitted if it is used by only one triangle (boundary of an open shape), by more than two triangles, or if its two triangles are *not coplanar*: $|n_1 \cdot n_2| < 0.99999$.

Consequently quad diagonals and cap radials (coplanar neighbours) disappear and only the true edges remain: a cube shows exactly its 12 edges, a hexagon its 6 sides, the arrow its 7 outline edges. Curved surfaces (sphere, torus, cone side) keep their mesh lines. The resulting counts are the E column in Table 1: e.g. cylinder $3N$ (two rims and N vertical lines), cone $2N$, sphere $S(2K - 1)$.

3.2.11 Normals, colors and texture coordinates

- **Normals:** analytic for sphere, torus, cylinder / cone, flat per face for cube / tetrahedron / prism, analytic gradient for surfaces, area-weighted vertex averages for imported models that have no normals.
- **Vertex colors** (used by Gouraud mode): by default a rainbow computed from the normalised position inside the bounding box, $t = \text{mean}(\frac{x-x_{\min}}{\Delta x}, \frac{y-y_{\min}}{\Delta y}, \frac{z-z_{\min}}{\Delta z})$ over the non-flat axes and hue = $0.83t$; optionally a two-color gradient mix(A, B, t) chosen in the GUI. For surfaces t is the height.

- **UV:** sphere and torus by their parameters (j/S etc.), cylinder / prism by angle and height, caps by a circle mapped to $[0, 1]^2$, cube and tetrahedron per face, 2D shapes by their bounding box.

3.3 Surface $z = f(x, y)$ and Imported Models

3.3.1 Mathematical surface

The user types a formula such as `sin(x)*cos(y)` and a range $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$ with a grid of $N \times N$ cells (default $N = 64$, up to 400).

Formula parser. `ExprParser.cpp` is a recursive-descent parser that compiles the text into a small expression tree evaluated for every grid point. Grammar (highest precedence last):

$$\begin{aligned} \text{expr} &::= \text{term} ((+|-) \text{term})^* & \text{unary} &::= (-|+) \text{unary} \mid \text{power} \\ \text{term} &::= \text{unary} ((*|/) \text{unary})^* & \text{power} &::= \text{primary} (\wedge \text{unary})? \\ \text{primary} &::= \text{number} \mid x \mid y \mid \pi \mid e \mid \text{name}(\text{expr}[, \text{expr}]) \mid (\text{expr}) \end{aligned}$$

Power is right-associative and binds tighter than unary minus ($-x^2 = -(x^2)$, $2^{3^2} = 512$). Implicit multiplication (`2x`, `3sin(x)`) and a leading `z=` are accepted. Functions: `sin cos tan asin acos atan sinh cosh tanh sqrt abs exp log ln floor ceil` and the two-argument `pow atan2 min max`. Errors (unknown name, missing parenthesis, wrong argument count) are reported in the GUI together with the position and the previous mesh is kept.

Vertices. For $i, j = 0..N$: $x_i = x_{\min} + i \Delta x$, $y_j = y_{\min} + j \Delta y$, $z_{ij} = f(x_i, y_j)$. The surface is centred in x, y and uniformly scaled by $s = 1 / \max(\Delta X, \Delta Y)$ so that its footprint is about one unit wide (it can be scaled like any other object). The mathematical z -axis becomes world “up” ($+Y$) and the mathematical y becomes $-Z$ so that the coordinate system stays right-handed:

$$p_{ij} = (s(x_i - c_x), s z_{ij}, -s(y_j - c_y)), \quad n_{ij} = \text{normalize}(-f_x, 1, f_y),$$

where f_x, f_y are central differences of f ($h = 10^{-4}$ of the range), and $(u, v) = (i/N, j/N)$. $V = (N + 1)^2$, stored at index $a = j(N + 1) + i$.

Indices. For every cell with corners $a = (i, j)$, $b = (i+1, j)$, $c = (i, j+1)$, $d = (i+1, j+1)$ the triangles are (a, b, c) and (b, d, c) (counter-clockwise seen from $+Y$). A cell is skipped when any corner has f non-finite or $|f| > 10^3$ (e.g. $\log x$ for $x \leq 0$, $1/x$ at 0), so $I = 6N^2$ for a fully finite function and less otherwise. If no cell is finite the surface is rejected with an error message.



3.3.2 Importing .obj and .ply models

Both loaders are written by hand (no external model library) and fill the same `MeshData`.

OBJ. Lines `v`, `vt`, `vn` fill three arrays; each `f` corner `v/vt/vn` (any of `v`, `v/vt`, `v/vn`, `v/vt/vn`; negative indices allowed) is mapped to a GPU vertex. A vertex is the *unique triple* (position, uv, normal) — a lookup table reuses an existing vertex when the same triple appears again — so a vertex shared by many faces is stored once, but a vertex with two different normals or UVs is split. A face with k_f corners becomes a triangle fan (f_0, f_i, f_{i+1}), $i = 1..k_f - 2$, thus $I = 3 \sum_f (k_f - 2)$. (Fan triangulation is exact for the convex faces found in typical models.)

PLY. The header is parsed generically (`ascii`, `binary_little_endian`, `binary_big_endian`; all scalar types; list properties). For the `vertex` element the properties `x y z`, `nx ny nz`, `s t/u v`, `red green blue` are used, and for `face` the list `vertex_indices` is fan-triangulated as above. Unknown elements and properties are skipped.

Post-processing. The model is re-centred on its bounding-box centre and scaled so that its largest extent is 1; normals are generated (area-weighted, merged by position) if the file has none; spherical UV are generated if it has none; vertex colors are kept when present (PLY) and rainbow otherwise; finally the wire edge list is built. Samples (`assets/models/`): `torusknot.obj` (3417 vertices, 6400 triangles, with normals and UV), `octahedron.obj` (6 vertices, 8 triangles, positions only), `icosphere_colored.ply` (`ascii`, 42 vertices, 80 triangles, colors) and `cube_binary.ply` (`binary`, 8 vertices, quads $\rightarrow$ 12 triangles).

3.4 Rendering Modes and Shaders

All objects are drawn by *one* shader program (`shapes.vert`, `shapes.frag`). The integer uniform `mode` chosen per object selects the mode:



mode	Render mode	Color computation
0	Flat color	<code>objectColor</code> , no lighting (a single uniform color).
1	Gouraud	Lighting evaluated per <i>vertex</i> in the vertex shader from the vertex color; the resulting color is interpolated across the triangle.
2	Phong	Normal interpolated; lighting evaluated per <i>fragment</i> with <code>objectColor</code> .
3	Texture	Same lighting as Phong, but the albedo is <code>texture(tex, uv*uvScale)</code> .
4	Wireframe	<code>objectColor</code> , drawn with <code>GL_LINES</code> from <code>wireIndices</code> .
5	(internal)	Unlit vertex color: ground grid and axes.

Lighting model. Modes 1–3 use the Phong reflection model of Section 2.3.3 with one point light. In Part 1 the material is reduced to three numbers per object: the *albedo* c (the object color, the vertex color or the texel), which serves as both m_a and m_d , the *Specular* slider (m_s is a gray value) and the *Shininess* (n). The lighting is *two-sided*: if $N \cdot V < 0$ the normal is flipped, so the back of a 2D shape and meshes with inconsistent winding are lit correctly (face culling is disabled).

Listing 1: Core of the lighting function (both shaders).

```

1  vec3 phong(vec3 N, vec3 P, vec3 albedo) {
2      vec3 V = normalize(viewPos - P);
3      if (dot(N, V) < 0.0) N = -N;           // two-sided lighting
4      vec3 L = normalize(light.position - P);
5      vec3 ambient = light.ambient * albedo;
6      float diff = max(dot(N, L), 0.0);
7      vec3 diffuse = light.diffuse * diff * albedo;
8      vec3 R = reflect(-L, N);
9      float spec = diff > 0.0 ? pow(max(dot(V, R), 0.0), shininess) : 0.0;
10     return ambient + diffuse + light.specular * spec * specularStrength;
11 }

```

Gouraud vs. Phong. In Gouraud mode [3] `phong()` is called in the *vertex* shader with the vertex color as albedo and the result is passed as an interpolated out variable; the fragment shader only outputs it. In Phong mode the vertex shader passes the world position and normal, and the fragment shader calls `phong()` for every pixel. The difference is visible in the highlight on low-resolution meshes: Gouraud can miss or smear a highlight that falls inside a triangle.

Highlights on flat faces. A curved surface always has some point whose normal reflects the light straight to the eye, so a sphere or torus shows a bright spot. A flat face has a single normal, hence a single reflection direction: the highlight is either one broad bright area (if the mirror point of the light falls on the face) or absent (if it falls outside

the face, as with the default light on the cube in Figure 2a). With the light placed so that the mirror point lies on the top face (Figure 2b) the cube shows its highlight. This is the expected behaviour of the Phong model and not a rendering error; lowering the shininess widens the highlight.

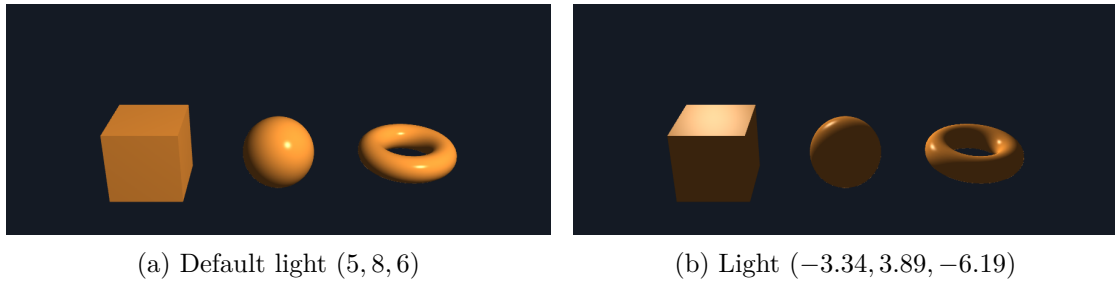
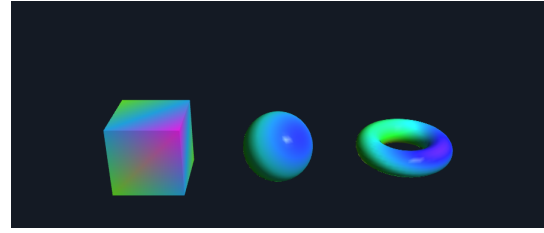


Figure 2: Phong highlight on a flat face depends on the light position (same objects, same camera).

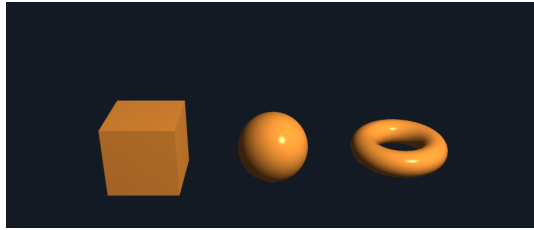
Texture. Images are loaded with `stb_image` (rows flipped so that $v = 0$ is the bottom), uploaded as RGBA8 with generated mipmaps, `GL_REPEAT` wrapping and trilinear filtering. A *Tiling* slider multiplies the UV. If no image is chosen, a built-in checkerboard is used.



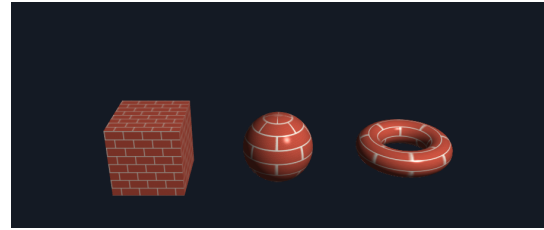
(a) Flat color



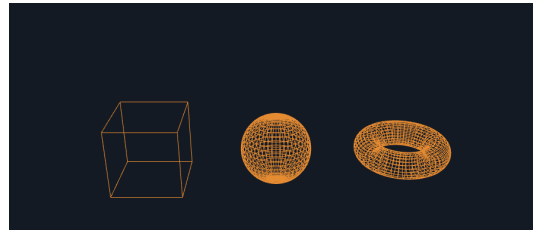
(b) Gouraud (rainbow vertex colors)



(c) Phong



(d) Texture (bricks.png)



(e) Wireframe

Figure 3: The five render modes on a cube, a sphere and a torus.

3.5 Object Transformations

Each object has a translation t , Euler rotation angles $(\theta_x, \theta_y, \theta_z)$ in degrees and a (non-uniform) scale s . The model matrix is

$$M = T(t) R_z(\theta_z) R_y(\theta_y) R_x(\theta_x) S(s),$$

i.e. a vertex is first scaled, then rotated about X , Y , Z and finally translated. M is uploaded as the `model` uniform, and normals are transformed with the normal matrix $(M^{-1})^T$ (upper 3×3), which stays correct under non-uniform scaling. The inspector exposes these as sliders and shows the current 4×4 matrix. *Ctrl + left drag* in the viewport rotates the selected object, and *Reset transform* returns it to the centre.

3.6 User Interface and User Instructions

3.6.1 Using the application

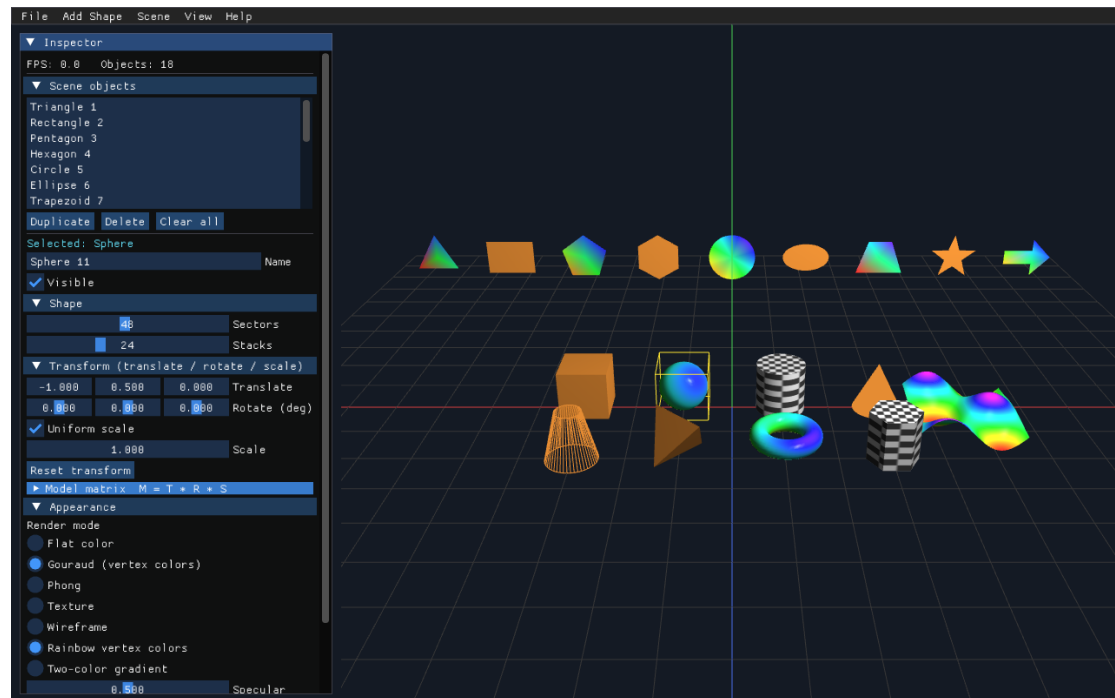


Figure 4: The interface: menu bar (top), inspector (left) and the 3D viewport with the demo scene. The yellow box marks the selected object.

- **Add Shape** menu: choose a 2D shape, a 3D shape, $Surface\ z = f(x, y)$ or *Import model....* The new shape appears at the centre and becomes selected.
- **Inspector** → *Scene objects*: select, duplicate, delete or clear objects. *Shape*: parameters of the selected shape (segments, star points, inner ratio, top radius, tube radius, prism sides, rectangle size, ...). For a surface, the formula, presets, ranges and resolution.
- *Transform*: translate / rotate / scale; *Appearance*: render mode, color, specular and shininess, vertex-color scheme (Gouraud), texture image and tiling.
- **Scene** menu: delete / duplicate, clear, load a demo scene with all shapes, set all objects to one mode. **View** menu: reset camera, front (2D) view, top view, grid, axes. **File**: import a model, save a screenshot.
- **Lighting** (light position and colors) and **Camera / View** (field of view, distance, background) at the bottom of the inspector.

Input	Action
Left mouse drag	orbit the camera
Right mouse drag	pan
Mouse wheel	zoom
Ctrl + left drag	rotate the selected object
Arrow keys / Shift + arrows	orbit / pan
+ - or PageUp / PageDown	zoom
R	reset camera
W	toggle wireframe (selected object, or all if none is selected)
Del	delete the selected object
F12	save a screenshot (screenshot_<time>.png)
H / Esc	help window / quit

Command line (for scripted screenshots). All figures in this report were produced with options such as

```
./build/shapes --add star --add torus --mode phong --screenshot out.png
```

Available options: --demo, --add <shape>, --mode flat|gouraud|phong|texture|wire, --surface "<formula>", --model <file>, --texture <png>, --view front|top, --distance d, --select i, --no-grid, --with-gui.

3.7 Results

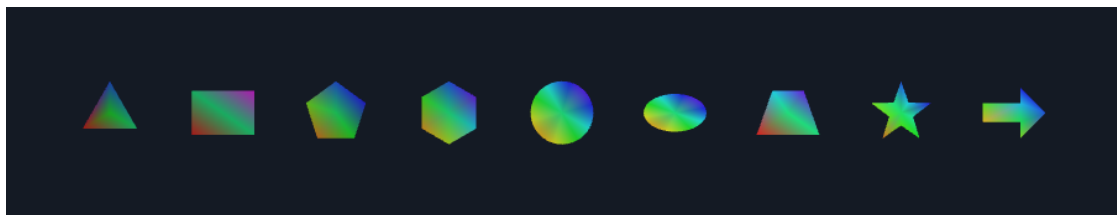


Figure 5: 2D shapes (front view, Gouraud with rainbow vertex colors): triangle, rectangle, pentagon, hexagon, circle, ellipse, trapezoid, star, arrow.

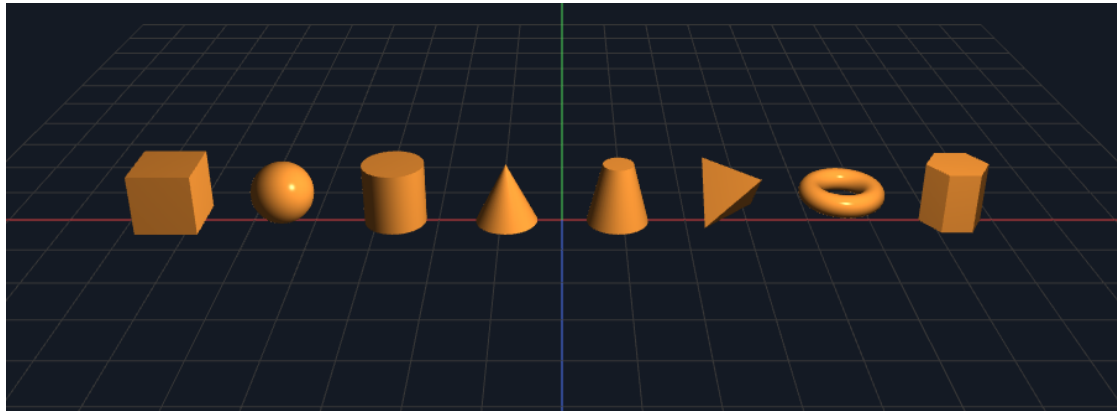
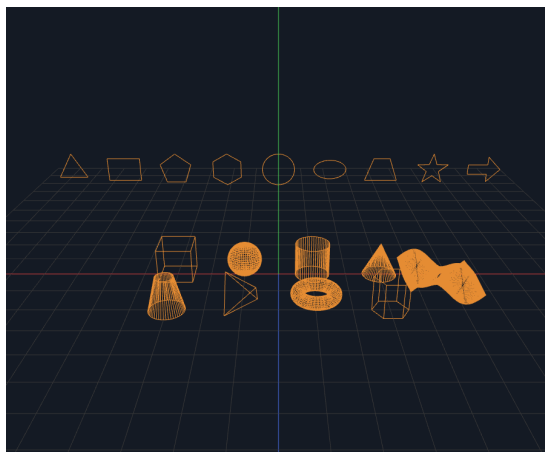
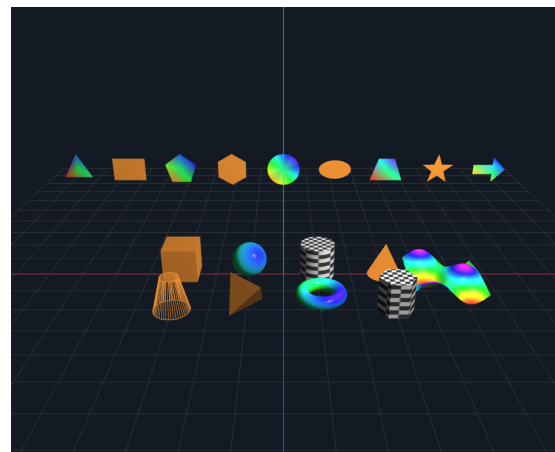


Figure 6: 3D solids (Phong): cube, sphere, cylinder, cone, truncated cone, tetrahedron, torus, prism.

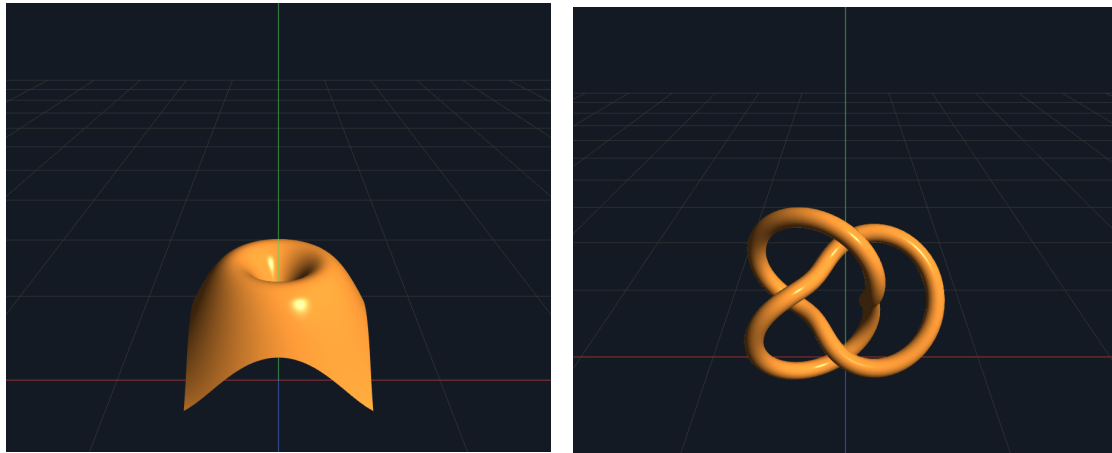


(a) All shapes in wireframe: only the true edges



(b) Demo scene: a mix of render modes

Figure 7: Wireframe and mixed-mode scenes.



(a) $z = 2 \sin \sqrt{x^2 + y^2}$, range $[-3, 3]^2$

(b) Imported `torusknot.obj`

Figure 8: A mathematical surface and an imported model, both in Phong mode.

3.8 Testing

`make test` builds `tests/test_core.cpp`, which needs no GPU and checks:

- the formula parser: precedence, right-associative power, unary minus, implicit multiplication, case-insensitivity, the `z=` prefix, and nine malformed inputs that must be rejected;
- every shape generator: all indices are in range, all normals are finite and unit length, colors are in $[0, 1]$, the wire list is non-empty, every triangle is wound consistently with its normal (counter-clockwise from outside), and the exact number of edges of the planar shapes and solids (e.g. cube 12, hexagon 6, arrow 7, prism $3n$);
- surface building (valid, partly non-finite, fully non-finite, syntax error, empty range);
- the four sample models: loads correctly, is centred and fits a unit box, binary and ascii PLY, PLY colors preserved, missing file and unsupported extension rejected.

All 191 199 checks pass. The closed-form counts of Table 1 were additionally verified against the generators by a separate program for all shapes at the defaults and for many parameter combinations.

3.9 Limitations

ImGui has no native file dialog, so models and textures are chosen from the `assets/` folders or by typing a path; objects are selected from the list (there is no picking by mouse



click); OBJ faces are fan-triangulated, which assumes convex faces; material files (.mtl) are ignored; Gouraud shading interpolates the lit vertex color, so highlights depend on mesh resolution (this is intended and shown in the comparison with Phong).

4 Part 2: Visualization of Atoms and Molecules

4.1 Architecture and Components

4.2 Visualizing Atoms: Bohr's Model

4.3 Visualizing Molecules: Ball-and-Stick Model

4.4 Animation (Optional)

4.5 Rendering and Lighting

4.6 User Interface and User Instructions

4.7 Results

4.8 Limitations

5 Conclusion

Both applications are built on the same OpenGL 3.3 core-profile foundation and on the shared code in `common/` (shader loading, orbit camera, ImGui, GLM), so the two parts are independent programs that reuse the same graphics infrastructure.

5.1 Part 1

Part 1 draws all required 2D and 3D shapes, a user-defined surface and imported .obj / .ply models; supports the five render modes (flat, Gouraud, Phong, texture, wire-frame); lets the user translate, rotate and scale every object and navigate with mouse and keyboard; and is organised in small modules of which the geometry, parsing and loading parts are independent from OpenGL and tested automatically.



5.2 Part 2

6 Source Code

<https://github.com/<account>/<repository>> (folders `part1_shapes`, `part2_molecules` and `common`).



References

- [1] M. Segal and K. Akeley, *The OpenGL Graphics System: A Specification, Version 3.3 (Core Profile)*, Khronos Group.
- [2] B. T. Phong, “Illumination for Computer Generated Pictures,” *Communications of the ACM*, vol. 18, no. 6, pp. 311–317, 1975.
- [3] H. Gouraud, “Continuous Shading of Curved Surfaces,” *IEEE Transactions on Computers*, vol. C-20, no. 6, pp. 623–629, 1971.
- [4] GLFW – an OpenGL library for windows, input and contexts. <https://www.glfw.org>
- [5] GLEW – the OpenGL Extension Wrangler Library. <https://glew.sourceforge.net>
- [6] G-Truc Creation, OpenGL Mathematics (GLM). <https://github.com/g-truc/glm>
- [7] O. Cornut, Dear ImGui – bloat-free graphical user interface for C++. <https://github.com/ocornut/imgui>
- [8] S. Barrett, stb single-file public domain libraries (stb_image, stb_image_write). <https://github.com/nothings/stb>